P.Vishwa Nandan(192473015)

Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal

1. Introduction

An online examination portal allows students to submit their answer files before the examination deadline.

However, a submission may fail due to different conditions:

1. The examination time has expired.
2. The student is not registered for the course.
3. The answer file exceeds the allowed size limit of 500 KB.
4. The storage system encounters an error while saving the answer file.

Java provides user-defined exceptions to handle application-specific errors.

In this system, custom exceptions are created and propagated through three layers:

- Controller Layer
- Service Layer
- Storage Layer

Exception chaining is also implemented to preserve the original cause of an error.

2. Exception Hierarchy Design

A user-defined exception is an exception created by the programmer to represent a specific application error.

The examination portal uses a parent exception called ExamException.

The following custom exceptions are designed:

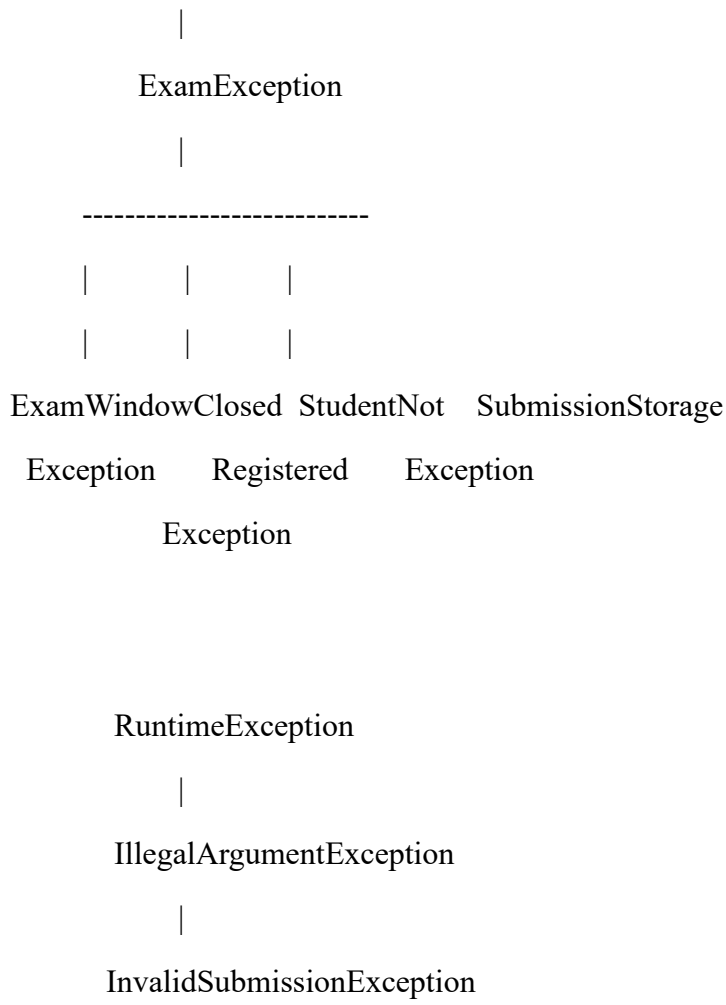
1. ExamWindowClosedException – Occurs when the examination deadline has passed.
2. StudentNotRegisteredException – Occurs when a student is not registered for the course.
3. SubmissionStorageException – Occurs when the answer file cannot be stored.
4. InvalidSubmissionException – Occurs when the submission contains invalid input.

Exception Hierarchy Diagram

Throwable

|

Exception



3. Checked and Unchecked Exceptions

Java exceptions are classified into checked and unchecked exceptions.

Checked Exceptions

Checked exceptions are checked by the compiler.

They must be handled using try-catch or declared using throws.

The following exceptions are checked:

- ExamException
- ExamWindowClosedException
- StudentNotRegisteredException
- SubmissionStorageException

These exceptions represent important examination business rules and storage failures that the application must handle.

Unchecked Exceptions

Unchecked exceptions occur during program execution.

They do not require mandatory handling or declaration.

InvalidSubmissionException extends IllegalArgumentException and is used for invalid submission parameters.

Classification Table

Exception	Type	Reason
ExamException	Checked	Base exception for examination failures
ExamWindowClosedException	Checked	Submission is not allowed after the deadline
StudentNotRegisteredException	Checked	Student registration must be verified
SubmissionStorageException	Checked	Storage failure must be handled
InvalidSubmissionException	Unchecked	Represents invalid programming or input conditions

4. Exception Propagation Across Three Layers

The examination portal follows a three-layer architecture.

A. Controller Layer

The controller receives the student's submission request.

Responsibilities:

- Accept student details.
- Receive the answer file.
- Call the service layer.
- Handle exceptions and display appropriate messages.

B. Service Layer

The service layer contains the examination business logic.

Responsibilities:

- Check whether the examination is open.
- Verify student registration.
- Validate the answer file size.
- Call the storage layer.

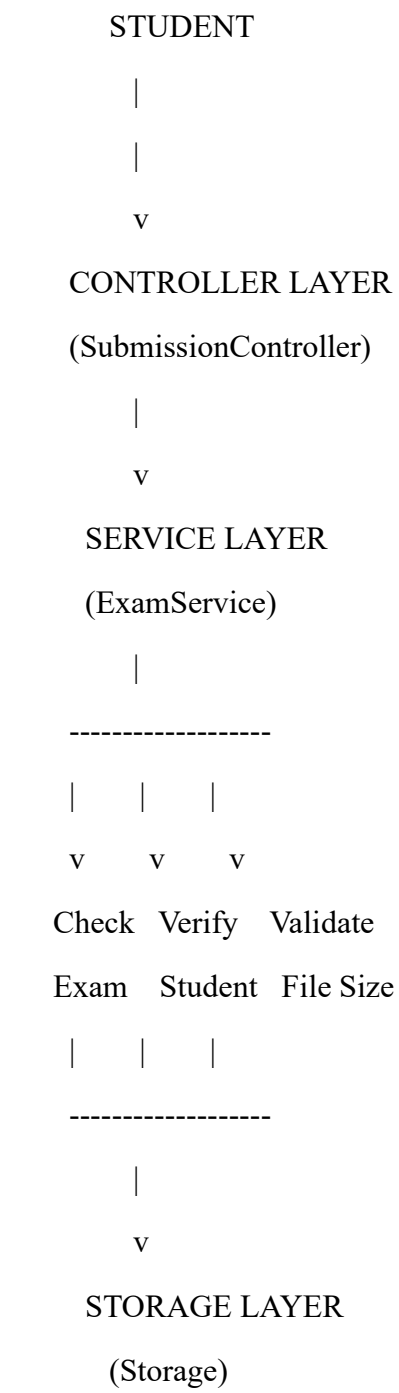
C. Storage Layer

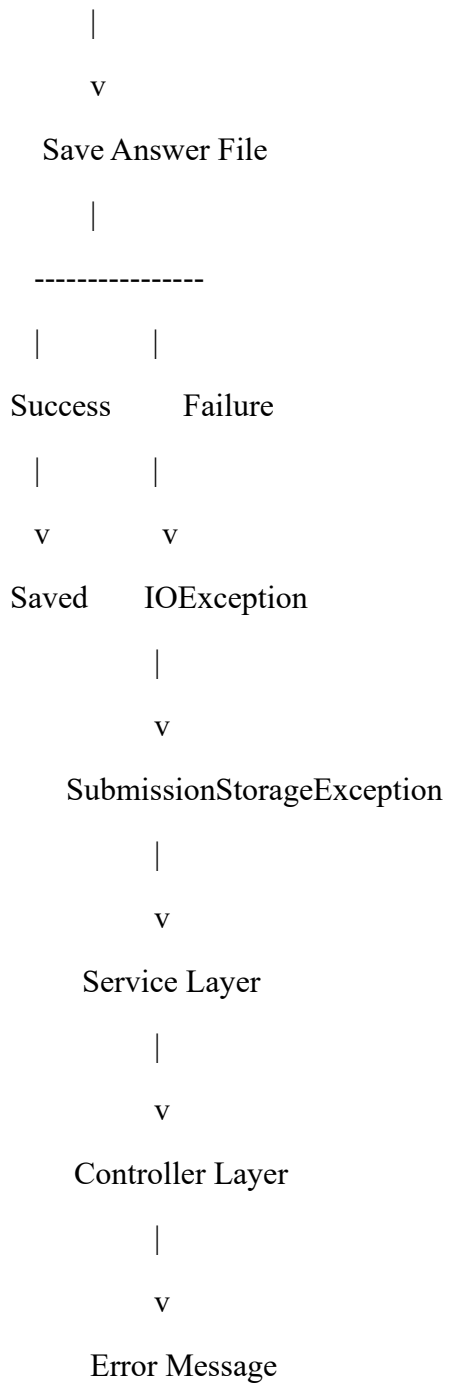
The storage layer saves the answer file.

Responsibilities:

- Store the answer file.
- Handle file-storage errors.
- Convert IOException into SubmissionStorageException.
- Preserve the original exception using exception chaining.

Architecture Diagram





5. Exception Propagation

Exception propagation means passing an exception from one method or layer to another until it is handled.

Example:

1. The storage layer encounters an IOException.
2. It converts the IOException into SubmissionStorageException.
3. The service layer allows the exception to propagate.

4. The controller catches the exception.
5. The controller displays an appropriate error message to the student.

This approach separates error handling from business logic.

6. Complete the Code – Exception Chaining

The assignment provides an incomplete program.

TODO 1: Wrap the IOException in a SubmissionStorageException while preserving the original IOException as its cause.

7. Expected Output

Saved answer01.pdf

Error: Could not store answer02.pdf

Cause: Disk quota exceeded

Explanation of Output

First Submission:

storage.save("answer01.pdf", 120);

The file size is 120 KB, which is below the 500 KB limit.

Therefore, the file is saved successfully.

Second Submission:

storage.save("answer02.pdf", 800);

The file size is 800 KB, which exceeds the allowed limit.

Therefore, an IOException is generated.

The catch block converts it into SubmissionStorageException.

The original IOException is preserved as the cause.

The controller catches the exception and displays:

Error: Could not store answer02.pdf

Cause: Disk quota exceeded

8. Exception Chaining Using getCause()

Exception chaining means attaching the original exception to a newly created exception.

Example:

throw new SubmissionStorageException(

"Could not store " + fileName, e

);

Here:

- The first argument contains the new exception message.
- The second argument contains the original IOException.

The getCause() method retrieves the original exception.

Example:

System.out.println(e.getCause().getMessage());

Output:

Disk quota exceeded

Importance of Exception Chaining

1. Preserves the original error information.
2. Helps identify the actual cause of failure.
3. Makes debugging easier.
4. Provides meaningful application-level error messages.
5. Maintains the relationship between different exceptions.

9. What Happens If the Original Cause Is Not Passed?

Consider the following incorrect implementation:

```
catch (IOException e) {

   throw new SubmissionStorageException(
         "Could not store " + fileName, null
   );
}
```

Here, the original IOException is not attached to the new exception.

Therefore:

e.getCause()

returns null.

If we execute:

System.out.println(e.getCause().getMessage());

A NullPointerException occurs because the cause is null.

Therefore, the output will not display:

Cause: Disk quota exceeded

Instead, the program encounters another exception.

Conclusion: Passing the original exception to the constructor is important because it preserves the actual reason for failure.

10. Failure Scenarios and Testing

The examination portal must handle different submission failures.

Test Case	Scenario	Expected Exception	Expected Result
1	Exam deadline expired	ExamWindowClosedException	Submission rejected
2	Student not registered	StudentNotRegisteredException	Submission rejected
3	File exceeds 500 KB	SubmissionStorageException	Storage error displayed
4	Storage system fails	SubmissionStorageException	Original cause preserved
5	Invalid file size	InvalidSubmissionException	Invalid input rejected
6	Valid submission	No exception	Submission successful

11. Demonstrating the Chained Stack Trace

A stack trace provides information about where an exception occurred and how it propagated through the application.

The following example prints the complete exception chain.

Add this statement inside the catch block:

e.printStackTrace();

Example Stack Trace

SubmissionStorageException:

Could not store answer02.pdf

at Storage.save(StorageDemo.java:...)

```
    at StorageDemo.main(StorageDemo.java:...)


Caused by: java.io.IOException:
Disk quota exceeded


    at Storage.save(StorageDemo.java:...)
```

The exact line numbers depend on the source file.

Explanation

1. SubmissionStorageException represents the application-level storage failure.
2. Could not store answer02.pdf explains which file could not be stored.
3. at Storage.save() identifies where the new exception was generated.
4. at StorageDemo.main() identifies the method that called the storage operation.
5. Caused by: IOException identifies the original low-level exception.
6. Disk quota exceeded explains the actual reason for the failure.

The chained stack trace helps developers identify both the application-level failure and its original cause.

12. Where Is the Exception Finally Handled?

The exception is finally handled in the controller layer.

The controller catches ExamException and displays a meaningful message.

Example:

```
try {


    storage.save("answer02.pdf", 800);


}


catch (ExamException e) {


    System.out.println("Submission failed: " + e.getMessage());
```

}

Expected output:

Submission failed: Could not store answer02.pdf

The technical details of the original IOException can be logged separately for debugging.

The student receives a simple message rather than a low-level system error.

STUDENT A – Exception Hierarchy and Propagation Design

13. Reason for Choosing Checked and Unchecked Exceptions

Checked exceptions are used for important examination and storage failures because the application must explicitly handle them.

For example:

- ExamWindowClosedException prevents late submissions.
- StudentNotRegisteredException prevents unauthorised submissions.
- SubmissionStorageException identifies storage failures.

Unchecked exceptions are used for invalid submission parameters that should normally be prevented through validation.

This design makes exception handling more organised and predictable.

14. Propagation Path

The propagation path of a storage exception is:

Storage Layer

|

v

IOException

|

v

SubmissionStorageException

|

v

Service Layer

|

v

Controller Layer

|

v

Catch Block

|

v

Display Error Message

The storage layer converts the low-level IOException into a meaningful custom exception.

The exception travels through the service layer and is finally handled by the controller.

15. Message Displayed to the Student

The student should not receive technical details such as file paths, disk permissions or internal system errors.

Instead, the portal displays a user-friendly message.

Example:

Your answer could not be submitted.

Please try again later.

The original technical exception is retained for developers and system administrators.

STUDENT B – Failure Scenarios and Chaining Evidence

16. Demonstrating Exception Chaining

Exception chaining preserves the original exception using the constructor:

super(message, cause);

The original exception can be retrieved using:

getCause();

If the cause is not passed, the original failure information is lost.

Comparison

Feature	**With Exception Chaining**	**Without Exception Chaining**
Original exception	Preserved	Lost
getCause()	Returns IOException	Returns null

Debugging	Easier	More difficult
Stack trace	Shows both exceptions	Shows only the new exception
Error identification	Accurate	Limited

17. Observed Behaviour Table

Test	Input	Exception Generated	Cause Preserved
1	120 KB file	No exception	Not required
2	800 KB file	SubmissionStorageException	Yes
3	Missing storage resource	SubmissionStorageException	Yes, if wrapped
4	Exception without cause	SubmissionStorageException	No

JOINT DESIGN QUESTION

18. Why Is SubmissionStorageException Better Than IOException?

It is better for the storage layer to throw a meaningful SubmissionStorageException rather than allowing an IOException to travel unchanged to the user interface.

Reasons:

1. **Meaningful Error:** SubmissionStorageException clearly identifies that the answer submission could not be stored.
2. **Layer Separation:** The storage layer hides low-level implementation details from the controller.
3. **Easy Debugging:** Exception chaining preserves the original IOException.
4. **Better Maintenance:** Developers can identify storage-related failures easily.
5. **User-Friendly Messages:** The controller can display simple messages to students instead of technical system errors.

Therefore, custom exceptions improve maintainability, readability and error handling.

ALTERNATIVE CONCEPT AND COMPLEXITY ANALYSIS

19. Alternative Approach – Unchecked Custom Exceptions

In the original implementation, ExamException extends Exception.

Therefore, it is a checked exception.

If ExamException instead extends RuntimeException, it becomes an unchecked exception.

Example:

```
class ExamException extends RuntimeException {

    ExamException(String message) {
        super(message);
    }

    ExamException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Changes in Method Signatures

Original checked exception:

```
void save(String fileName, long sizeKb)
        throws SubmissionStorageException
```

Unchecked exception:

```
void save(String fileName, long sizeKb)
```

The throws declaration is no longer mandatory.

Changes in Calling Code

With checked exceptions, the calling method must handle the exception or declare it.

With unchecked exceptions, the calling method is not forced by the compiler to handle or declare it.

However, try-catch can still be used.

Example:

```
try {

    storage.save("answer02.pdf", 800);

}
```

```
catch (ExamException e) {


   System.out.println(e.getMessage());
}
```

This code works with both checked and unchecked exceptions.

Comparison

Feature	**Checked Exception**	**Unchecked Exception**
Parent class	Exception	RuntimeException
Compiler checking	Yes	No
throws declaration	Required or exception must be handled	Not required
Calling code	Must handle or declare	Optional
Error handling	Enforced by compiler	Programmer's responsibility

20. Time and Space Complexity of Exception Handling

Let d represent the depth of the method-call stack.

A. Time Complexity

When an exception is created and thrown, Java captures stack-trace information.

The stack-trace capture generally requires time proportional to the stack depth.

Therefore:

Time Complexity: O(d) for capturing stack-trace information.

Exception creation and stack-trace processing introduce additional overhead compared with returning a simple error code.

B. Space Complexity

An exception stores:

- Exception message.
- Exception type.
- Stack-trace information.
- Original cause, if available.

The stack-trace information requires approximately O(d) space.

When exception chaining is used, the original exception is also retained.

Therefore, memory usage increases because both exception objects and their associated stack-trace information remain reachable.

For a chain of k exceptions, with stack depths d_1, d_2, ..., d_k, the total retained stack-trace storage is approximately:

$O(d_1 + d_2 + ... + d_k)$.

C. Advantages of Checked Exceptions

1. The compiler ensures that important exceptions are handled or declared.
2. Calling methods are aware of possible failures.
3. Important business-rule failures are less likely to be ignored accidentally.

D. Advantages of Unchecked Exceptions

1. Method signatures are simpler.
2. Less exception-handling code is required.
3. Suitable for programming errors and invalid method arguments.

E. Final Comparison

Measurement	**Checked Exception**	**Unchecked Exception**
Exception creation	Runtime overhead	Runtime overhead
Stack-trace capture	O(d)	O(d)
Space for stack trace	O(d)	O(d)
throws declaration	Required or handled	Optional
Cause chaining	Supported	Supported
Compiler enforcement	Yes	No

Changing an exception from checked to unchecked does not remove the runtime cost of creating the exception or capturing its stack trace.

21. Conclusion

User-defined exceptions provide an organised way to handle application-specific failures in an online examination portal. The exception hierarchy separates examination, registration and storage errors. Exception propagation allows failures to move through the controller, service and storage layers. Exception chaining preserves the original cause using getCause(). Checked exceptions provide compiler-enforced handling, while unchecked exceptions

provide flexibility. Overall, custom exceptions improve application reliability, debugging and maintainability.